

tp4

1-

```
class Motor:
    def __init__(self, cilindrada, combustible):
        self.cilindrada = cilindrada
        self.combustible = combustible

    def encender(self):
        return f"Motor de {self.cilindrada}cc encendido ({self.combustible})"

    def apagar(self):
        return "Motor apagado"

class Auto(Motor):
    def __init__(self, marca, modelo, cilindrada, combustible):
        super().__init__(cilindrada, combustible)
        self.marca = marca
        self.modelo = modelo

    def describir(self):
        return f"{self.marca} {self.modelo} - {self.encender()}"

# Ejemplo de uso
auto = Auto("Toyota", "Corolla", 1800, "nafta")
print(auto.describir())
print(auto.apagar())
```

Esto está mal porque no cumple con la regla “Es un...”, un auto no es un motor, sino que tiene un motor, tiene ruedas, tiene puertas, etc. Lo indicado en este caso sería trabajar con composición, con relaciones entre las clases de las partes del auto, no tiene sentido usar herencia, generamos un acoplamiento innecesario.

2-La herencia se aplica para compartir comportamiento, reutilizar código y no para compartir atributos, lo que se puede hacer como alternativa sería usar composición instanciando un objeto de otra clase como atributo, por ejemplo en el punto anterior en vez de herencia podría instanciarse como atributo un **self.motor=Motor(peso, cilindrada, combustible)** en la clase auto como atributo.

3-`class Archivo:`

```
def __init__(self, nombre, pesoEnMB):
    print("Creando Archivo genérico...")
    self.nombre = nombre
    self.pesoEnMB = pesoEnMB
```

`class ArchivoVideo(Archivo):`

```
def __init__(self, nombre, pesoEnMB, duracion):
    print("Creando Archivo de Video...")
    super().__init__(nombre, pesoEnMB)
    self.duracion = duracion
```

```
video = ArchivoVideo("pelicula.mp4", 700, 120)
print(f"Nombre: {video.nombre}, Peso: {video.pesoEnMB}MB, Duración: {video.duracion}min")
```

-En lo que se ve acá y en la consola podemos inferir que el orden en el que se ejecutan los constructores al instanciar un objeto de ArchivoVideo es primero la clase hija ArchivoVideo, que pinte "creando archivo de video" (para esto ya se ejecutó anteriormente el init de la misma) y luego llama al constructor de la clase padre con el "super". Vale aclarar que no es buena práctica colocar el "super().__init__" luego de imprimir, el super debería estar exactamente luego del constructor de la clase ArchivoVideo.

-Es obligatorio que aparezca el "super().__init__..." en la clase hija, porque sino la misma no sabe qué hereda de la superclase padre porque esos atributos nunca se inicializarían.

Si omitimos esto, muestra en consola el siguiente error: `AttributeError:`

`'ArchivoVideo' object has no attribute 'nombre'`. Básicamente nos dice que el objeto video de ArchivoVideo no tiene un atributo nombre, justamente porque sacamos el constructor que le hereda ese atributo de la clase padre.

4a)

	Visibilidad y alcance	Relacion con la herencia	Recomendación de uso
Private	Mínima. Solo es visible dentro de la clase donde se declaró	No son visibles para las clases descendientes y no pueden ser redefinidos	Se recomienda declarar siempre los atributos como privados para garantizar el encapsulamiento
Protected	Intermedia. Visible para la propia clase y sus descendientes	Los métodos son heredados y forman parte de la interfaz que las subclasses exponen al exterior	Debe usarse para métodos o propiedades que brinden un servicio exclusivamente a sus clases hijas
Public	Total. Accesible desde cualquier otra clase o parte de la aplicación En Java, incluye también acceso "friendly".	Los métodos son heredados y forman parte de la interfaz que las subclasses exponen al exterior	Se usa para constructores, destructores y métodos que definen el comportamiento público del objeto

b- Yo opino que rompe el encapsulamiento, porque la comunicación o contrato que establece una superclase padre con las hijas hace que sea accesible con las mismas. El encapsulamiento dice que los detalles de cada clase deben estar ocultos, pero con protected quedan expuestos a las clases hijas, entonces cada clase hija puede modificar directamente los atributos.

Lo mejor sería hacerlo privado en la clase padre e ir heredando el método getter y setter a las hijas para que puedan acceder y así evitar el acoplamiento y el problema de encapsulamiento.

5- Es Falsa, los conceptos están invertidos, override es justamente lo que describe como overload. El override es cuando una clase hija redefine un método, de la superclase con los mismos parámetros, el mismo nombre del método y devolver el mismo tipo de valor (int, float, str, any, etc) pero redefine el comportamiento del mismo. Mientras que overload sería definir el mismo método (con el mismo nombre) en la misma clase con diferentes comportamientos debido a que cada una tendría parámetros diferentes.

```
class Animal:
    def hacerRuido(self):
        print("...")

class Gato(Animal):
    def hacerRuido(self):
        print("Miau")

class Perro(Animal):
    def hacerRuido(self):
        print("Guau")

class Vaca(Animal):
    def hacerRuido(self):
        print("Muuu")
```

En este ejemplo no hay "super().__init__" porque no hay atributos en la clase padre para llamarla y heredarlos.

6-Declarar una clase como final (o sellada en otros lenguajes) significa que esa clase no puede ser heredada. Al marcar una clase con esta palabra clave, se impide que se puedan construir clases derivadas o subclases a partir de ella.

-Existen dos razones:

-Mejoras de eficiencia: Declarar una clase como final permite que el compilador introduzca mejoras de eficiencia en el código. Al saber que la clase no tendrá descendientes, el sistema puede optimizar el manejo de sus objetos.

-Control de funcionalidad: En el caso de String, esto limita la posibilidad de que otros programadores creen nuevos tipos de cadenas de caracteres con funcionalidades extendidas mediante herencia. Sin embargo, esto no impide extender su funcionalidad por otros medios, como la delegación o la creación de métodos estáticos que procesen objetos String.

A pesar de estas ventajas técnicas, se recomienda evitar declarar clases como finales por norma general, ya que esto restringe la reutilización y la extensión del código, que son pilares de la programación orientada a objetos

7-Los 3 problemas principales con la herencia son el fuerte acoplamiento/herencia frágil que van de la mano. Esto a su vez dificulta el mantener el programa funcionando con cambios a futuro que pueden afectar a diferentes clases a la vez. Otro problema sería el encapsulamiento, que (dependiendo el lenguaje) da la opción de utilizar protected, el cual rompe el encapsulamiento; o definir getters y setters para poder heredar cada atributo privado de la clase padre. Por último está la herencia múltiple problemática en lenguajes que la permiten, si dos clases padre tienen un método con el mismo nombre, ¿cuál hereda la hija? Esto se conoce como el problema del diamante. También vale la pena mencionar que tiene dificultad para testeo

8-Expliquen por qué estas soluciones son ineficientes, rígidas y no representan bien el mundo real.

Son ineficientes porque puede producirse un “problema del diamante” ,mencionado en el punto anterior, además estamos confundiendo rol con “es un” una persona que es dos cosas a la vez no puede existir dos veces, o dejar de existir como alumnodocente y luego existir como docente, qué pasa si cambia sus datos al volver a instanciar un objeto nuevo luego de “destruir” el anterior? qué pasa si pone un mail diferente, una dirección, un nro de celular en caso de que no se destruya el objeto y se encuentre duplicado?

Además al ser la clase fija cuando el objeto es instanciado, no se puede modificar sobre la marcha.

```
class Persona:
    def __init__(self, nombre, dni):
        print("Creando Persona...")
        self.nombre = nombre
        self.dni = dni

class Alumno(Persona):
    def __init__(self, nombre, dni, legajo):
        print("Creando Alumno...")
        super().__init__(nombre, dni)
        self.legajo = legajo

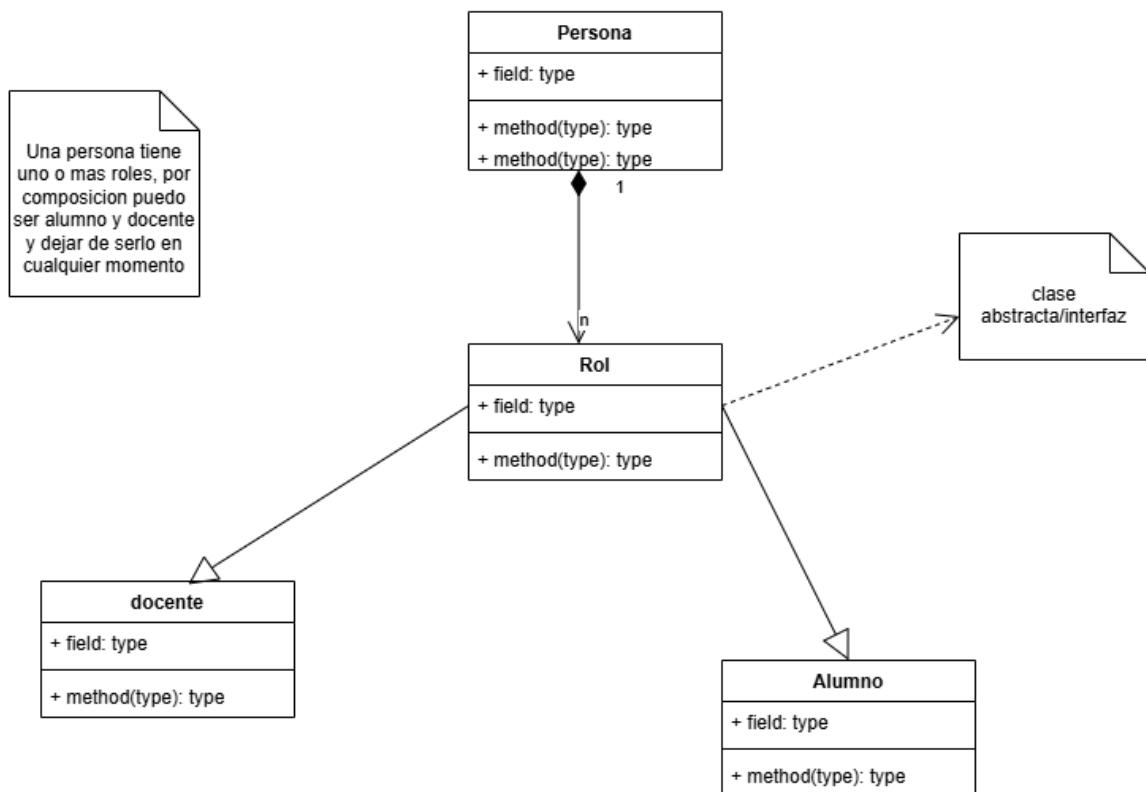
class Docente(Persona):
    def __init__(self, nombre, dni, cargo):
        print("Creando Docente...")
        super().__init__(nombre, dni)
        self.cargo = cargo

# Alumno avanzado contratado como docente auxiliar -> herencia múltiple
class AlumnoDocente(Alumno, Docente):
    def __init__(self, nombre, dni, legajo, cargo):
        print("Creando AlumnoDocente...")
        Alumno.__init__(self, nombre, dni, legajo)
        Docente.__init__(self, nombre, dni, cargo)

# Cuando se recibe, deja de ser Alumno y pasa a ser solo Docente
# Con herencia clásica, esto NO se puede modelar: la clase de un objeto
# no puede cambiar en tiempo de ejecución. Habría que crear un nuevo
# objeto Docente y "descartar" el AlumnoDocente, perdiendo la identidad
# original.

ad = AlumnoDocente("Juan", "12345678", "A-001", "Auxiliar")
print(ad.nombre, ad.legajo, ad.cargo)
```

9-



10-Se puede decir que: "t" hereda de "o" y "m"; "x" herada de "m" pero no de "o"; entonces m es clase padre, o puede ser una clase intermedia pero no es superclase.